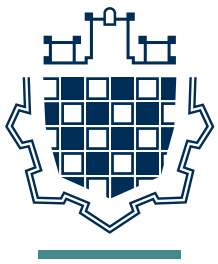

UNIVERSITY OF MANNHEIM

School of Business Informatics and Mathematics

Test Suite Minimization in LASSO Using Stimulus-Response Matrices

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

laith.sandouk@students.uni-mannheim.de

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering

School of Business Informatics and Mathematics

University of Mannheim

D – 68159 Mannheim

Phone: +49 621-181-3912, Fax +49 621-181-3909

Internet: http://swt.informatik.uni-mannheim.de

Abstract

Large-scale industrial software systems face significant challenges in executing regression test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [2]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case—test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [20]. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation within the *Large-Scale Software Observatorium* (LASSO) [12], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence; its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems—large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [1].

Our approach leverages *Stimulus–Response Matrices* (SRMs) [12], a two-dimensional representation mapping test stimuli to observed execution outcomes. We augment SRM entries with aggregate coverage and mutation information, enabling minimisation algorithms to operate on behavioural data rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along seven established criteria: coverage preservation, reduction effectiveness, fault-detection retention, mutation-coverage preservation, computational scalability, SRM compatibility, and implementation feasibility. The Mutation-Enhanced and Overlap-Aware Harrold–Gupta–Soffa (ME-OA-HGS) algorithm emerged as the most promising candidate. ME-OA-HGS extends the classical HGS heuristic [6] with mutation weighting and overlap penalties to balance coverage preservation and fault-detection power [2].

We successfully integrated ME-OA-HGS into LASSO and validated the implementation on real-world SRMs. Our evaluation achieved reductions within the ranges reported in prior studies—45–70 % test-suite reduction while maintaining $\geq 95\,\%$ statement coverage and $\geq 90\,\%$ mutation-score retention [6, 9]. The algorithm executes in $O(nm \log m)$ time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

List of Figures

List of Tables

1. Introduction

Software testing ensures program modifications do not introduce defects. As systems evolve, test suites expand, increasing execution cost and maintenance effort. Suites combine human-written, automatically generated, and AI-generated tests. Tools like Randoop [17] and EvoSuite [5] generate thousands of cases. In large projects, execution may require days or weeks [18]. While comprehensive suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, causing prolonged execution, increased costs, and elevated maintenance overhead.

Test-suite minimization systematically removes redundant tests while preserving fault-detection capability [20]. In automated and AI-assisted pipelines, minimization substantially reduces computational demand and feedback latency. However, excessive reduction risks eliminating unique defect-revealing tests. Effective minimization balances reduction with fault-detection retention.

1.1. Background and Motivation

Testing remains resource-intensive in software engineering. Studies estimate verification and validation consume up to half of development cost [3]. Although originating from traditional models, cost pressure persists in continuous-integration where automated or AI-generated suites may contain tens of thousands of tests [18, 2]. Executing all tests after each modification is often infeasible, motivating scalable approaches reducing suite size without compromising quality. Test-suite minimisation identifies and removes redundant tests while retaining those essential for coverage and fault detection.

1.2. LASSO Context

The Large-Scale Software Observatorium (LASSO) [12] provides infrastructure for static and dynamic software analysis at scale, featuring extensive executable systems from open-source repositories with domain-specific language for behavioral queries across thousands of systems.

LASSO employs *Stimulus–Response Matrices* (SRMs) [12] for systematic comparison. SRMs are two-dimensional matrices: rows represent test stimuli, columns represent systems, cells contain structured response objects capturing execution outcomes. From this representation, coverage metrics, mutation scores, and behavioral characteristics are systematically extracted and analyzed, enabling language-independent comparison of functionally equivalent implementations. Section 2.2 provides detailed SRM definitions and explains coverage/mutation extraction.

Despite LASSO's capabilities, the platform lacks integrated test-suite minimization. When analyzing hundreds or thousands of systems, redundant stimuli unnecessarily prolong execution and increase infrastructure costs. Integrating minimization would reduce test redundancy while preserving behavioral coverage necessary for cross-system comparison.

1.3. Problem Statement

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote a test suite and $R = \{r_1, r_2, \ldots, r_m\}$ denote coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. The *test-suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T) = \bigcup_{t_i \in T} C(t_i)$. This is equivalent to the minimum set-cover problem and is NP-complete [20], requiring heuristic algorithms [6] that balance reduction effectiveness, coverage preservation, and computational efficiency. Figure 1.1 illustrates this optimization.

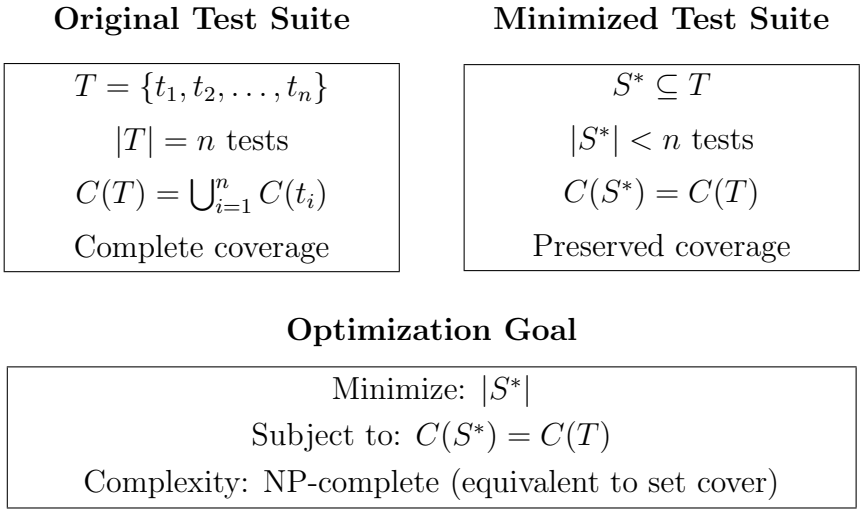

Figure 1.1.: The test-suite minimization problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test-suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test-suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

This thesis implements minimization directly on LASSO's SRM abstraction rather than integrating external tools. Existing frameworks (OpenClover, PIT) support limited languages and require source code access or language-specific instrumentation, conflicting with LASSO's language-independent design.

SRMs already encapsulate information necessary for behavioral minimization: test identifiers, execution results, coverage metrics, mutation data. Operating exclusively on this structure maintains language independence, ensures scalability by processing compact SRM representations rather than source artifacts, and promotes architectural cohesion by integrating seamlessly with LASSO's pipeline.

1.6. Thesis Structure

The remainder is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formalises the minimization problem.

Chapter 3 reviews related work across greedy, exact, search-based, data-driven techniques.

Chapter 4 defines evaluation criteria (C1–C7) and scoring framework for algorithm comparison under SRM constraints.

Chapter 5 applies the framework to rank algorithms and justify ME–OA–HGS selection.

Chapter 6 details implementation and LASSO integration.

Chapter 7 discusses limitations, trade-offs, deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation by introducing core concepts and formal definitions required for test-suite minimisation. We present metrics quantifying test-suite effectiveness, describe the *Stimulus–Response Matrix* (SRM) representation within LASSO, and formally define the minimisation problem.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [21]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program "parts" at different granularity levels [21]. Common coverage types in test-suite evaluation:

- **Instruction Coverage:** proportion of bytecode instructions executed (finest-grained, virtual machine level).
- **Line Coverage:** proportion of source lines executed (intuitive source-code feedback).
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes both execute.
- **Method Coverage:** proportion of methods invoked at least once.
- **Complexity Coverage:** proportion of independent execution paths exercised (cyclomatic complexity).
- **Class Coverage:** proportion of classes instantiated or whose static methods execute (coarse-grained).

Multi-level coverage captures execution at different granularities. Different types detect different fault classes [21]—high line coverage with low branch coverage may miss conditional faults; high method coverage without complexity coverage may miss execution paths in complex methods. Employing metrics across the spectrum from instruction-level to class-level provides complete quality assessment.

2.1.2. Mutation Score

Mutation testing quantifies test-suite effectiveness by measuring detection of artificially introduced faults (*mutants*) [10]. Each mutant is a slightly modified version (replacing operators, altering conditionals, modifying constants). Test suites execute against both original and mutated versions.

The *mutation score* expresses the proportion of mutants detected by the test suite and is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

A mutant is considered *killed* when a test produces a different observable outcome for the mutant than for the original program. This typically means that the test passes on the original system but fails when executed on the mutant. Conversely, if a test yields the same outcome on both versions, the mutant *survives*.

Mutation Types. Two main types of mutation testing are commonly distinguished [16]:

1. **Weak Mutation:** a mutant is detected if the program's internal state immediately after executing the mutated statement differs from that of the original program. Detection is based on local state divergence rather than final output.
2. **Strong Mutation:** a mutant is detected only if this internal divergence propagates to the program's externally observable behaviour, such as output values or test verdicts. This form requires execution to completion and comparison of final results.

Weak mutation focuses on local sensitivity to code changes, while strong mutation measures externally visible behavioural differences. Both flavours aim to approximate a test suite's fault-detection capability, with the mutation score serving as a quantitative indicator of test effectiveness.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO's central data structure for *Test-Driven Software Experimentation* [12], providing unified, language-independent format for recording and comparing run-time behaviour under controlled conditions. SRMs require two input artefacts: *Sequence Sheets* and *Actuation Sheets*.

2.2.1. Sequence Sheets and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test: tabular, human-readable, directly executable representations bridging code-based unit tests and tabular acceptance tests. Define linear method invocation sequences with input parameters and expected outputs. Notation is language-independent, focusing on behavioural semantics.

Typical columns:

- **Output:** expected result or observable state.
- **Operation:** operation/method name to invoke.
- **Service:** class to instantiate (`create`) or instance for invocation.
- **Input:** arguments or references to previously created objects.

Each row represents one stimulus (method invocation with inputs and expected output). Objects instantiated via `create`; cell references (e.g., `A1`, `D3`) reuse instances. Parameterised sheets introduce placeholders (e.g., `?p1`, `?p2`) substituted at runtime for test variants.

Actuation Sheets capture observed responses when executing sequence sheets. Preserve structure but replace expected outputs with actual runtime data, forming *actuation records*. Each record represents one (stimulus, system) execution with concrete inputs and outputs. Sequence sheets describe *intended behaviour*;

actuation sheets record *empirical behaviour*. Executing one sequence sheet across multiple systems produces several actuation sheets.

Stimulus (Sequence Sheet)

Out	Operation	Service	Input
create	`create`	`'Stack`	
–	`push`	A1	42
42	`pop`	A1	

Observed Behaviour (Actuation Sheet)

Out	Operation	Service	Input
`<instance>`	`create`	`'Stack`	
`true`	`push`	A1	42
`42`	`pop`	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to Stimulus–Response Matrices

LASSO generalises stimuli-to-actuation mapping into multidimensional *Stimulus–Response Matrices* (SRMs), aggregating actuation results of test sets executed across multiple systems/variants:

$$M = [M_{ij}] \quad \text{with} \quad M_{ij} = \text{Actuation}(t_i, v_j),$$

where $t_i \in T$ (stimulus/row), $v_j \in V$ (system variant/column), M_{ij} (actuation record with inputs, outputs, analysis attributes).

Core information: input parameters, outputs/return values, exceptions/failures, optional execution traces.

SRMs support *analysis attributes*—key–value pairs storing additional measurements: non-functional metrics (execution time, memory), coverage data (line, branch, method), mutation results/scores, static properties/complexity.

Flexible schema stores functional and non-functional behaviour via two representations:

1. **Black-box:** aggregated input/output data per stimulus.
2. **White-box:** full internal invocation structure for detailed trace analysis.

	Original	Mutant 1	Mutant 2	Mutant 3
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across variants. Behavioural discrepancies indicate killed mutants.

2.2.3. Coverage and Mutation Integration in SRMs

LASSO integrates coverage and mutation data into SRMs. JaCoCo measures structural coverage, reporting entity types (instruction, line, branch, method, class) and metrics (`COVEREDCOUNT`, `TOTALCOUNT`, `COVEREDRATIO`) [8]. Stored as analysis attributes under dedicated `sheetId` (e.g., `JACOCO`), enabling fine-grained SRMPATH retrieval.

Mutation analysis: each mutant treated as separate variant v_j. SRM cell M_{ij} contains mutant actuation. Mutant *killed* if response differs from original:

$$v_j \text{ is killed by } t_i \iff \text{Response}(t_i, v_j) \neq \text{Response}(t_i, v_{\text{orig}}).$$

Comparing responses enables data-driven mutation coverage computation:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100\,\%.$$

2.2.4. Interpretation and Applications

SRMs unify execution data (functional outcomes, non-functional metrics, coverage, mutation) into language-independent structure, facilitating regression testing, differential testing, N-version testing, mutation-based assessment. SRMs form LASSO's empirical foundation for optimisation tasks like test-suite minimisation and coverage preservation.

2.3. The Test-Suite Minimisation Problem

Section 1.3 introduced the optimisation goal. This section provides formal definition, computational complexity, and LASSO context.

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \ldots, t_n\}$ denote test suite (stimuli) and $R = \{r_1, r_2, \ldots, r_m\}$ coverage requirements. Each $t_i \in T$ covers subset $C(t_i) \subseteq R$. Full suite coverage:

$$C(T) = \bigcup_{i=1}^{n} C(t_i).$$

Test-suite minimisation problem seeks smallest subset $S^* \subseteq T$ preserving coverage:

$$S^* = \arg\min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T).$$

Equivalent to *minimum set-cover problem*.

2.3.2. Computational Complexity

Test-suite minimisation is NP-complete [20]; no polynomial-time algorithm guarantees optimal solutions for arbitrary instances. Practical implementations rely on approximation algorithms [4] and heuristics [6] trading exactness for scalability. Classical greedy achieves $O(\ln m)$ approximation ratio (m requirements).

2.3.3. Test-Suite Minimisation in the LASSO Context

Within LASSO, minimisation operates on SRM data. Each stimulus t_i corresponds to one SRM row; coverage and mutation attributes stored in response objects M_{ij}. Requirement set $C(t_i)$ derived from JaCoCo metrics and mutation results. Objective: select subset $S^* \subseteq T$ preserving full coverage:

$$C(S^*)_{\text{SRM}} = C(T)_{\text{SRM}}.$$

Result: reduced SRM containing only selected stimuli.

2.3.4. Algorithmic Categories

Following established classifications [20, 13, 2], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [6]), **exact optimisation** (ILP, constraint-satisfaction [14]), **search-based methods** (genetic algorithms [7]),

data-driven methods (clustering, overlap-aware [2]). Detailed explanation in Chapter 3.

3. Minimization Algorithms

This chapter surveys test-suite minimisation algorithms, organised by category: greedy heuristics, exact optimisation, search-based approaches, and data-driven techniques. For each category, we examine representative algorithms illustrating key characteristics and trade-offs.

3.1. Greedy Heuristics

Greedy heuristics iteratively select tests covering the most uncovered requirements until complete coverage is achieved [20, 13]. These algorithms operate in polynomial time with predictable performance. Refinements incorporate requirement rarity or overlap penalties to improve retention while maintaining efficiency [6, 2].

3.1.1. Classical Greedy

Classical greedy iteratively selects tests covering the most uncovered requirements until complete coverage [4, 20]. Time complexity is $O(nm)$ (n tests, m requirements) with $O(\ln m)$ approximation ratio, optimal for polynomial-time set-cover [4].

Empirical studies show 60–93 % reduction (median 67 %) maintaining 100 % structural coverage [15]. However, mutation scores drop 3.5–21 %: Apache projects lose 3.5–9.5 %, others 14–21 % [15], demonstrating structural coverage inadequately proxies test quality.

3.1.2. Harrold–Gupta–Soffa

HGS prioritises requirements by rarity [6], recognising tests covering rare requirements are more critical. Prioritising rare requirements early increases probability

each unique requirement is covered, reducing corrective additions. This preserves $O(\ln m)$ approximation while improving retention on instances with skewed cardinality distributions [4].

The algorithm operates in phases by cardinality: Phase 1 selects tests uniquely covering any requirement, Phase 2 processes cardinality = 2, continuing until complete coverage. Time complexity is $O(nm \log m)$ from cardinality sorting.

Empirical evaluation shows minimal improvement over classical greedy. HGS produces suites 0.37–2.92 % smaller [15]. Mutation differences (0.09–4.88 %) are statistically insignificant ($p \gg 0.05$) [15]. JavaScript applications show approximately 70 % reduction with preserved mutation coverage [9]. Performance differences fall within error margins, suggesting cardinality-based prioritisation provides limited benefit when overlap is moderate.

3.1.3. Delayed Greedy

Delayed greedy [19] uses concept lattices to eliminate implied redundancies before greedy selection. The lattice identifies subsumption: if test t_i covers all requirements of t_j plus additional requirements, t_j is redundant. After removing subsumed tests via lattice analysis, greedy selection proceeds on the reduced set. Three phases: object reduction, attribute reduction, greedy selection. Polynomial complexity, though exact bound unstated [19].

Delayed greedy produces suites equal to or smaller than classical greedy in 75 % of cases, equal to or smaller than HGS in 64 % [19, 20], guaranteeing 100 % coverage retention. JavaScript implementations achieve 65–74 % reduction maintaining mutation scores [9]. Faster runtime than classical greedy despite lattice overhead, suggesting redundancy elimination reduces greedy selection cost [9].

3.1.4. Mutation-Enhanced Overlap-Aware HGS

Recent empirical findings demonstrate that greedy minimisation techniques incorporating mutation coverage as a selection criterion achieve superior fault-detection retention compared to approaches relying solely on structural coverage metrics. Jehan and Wotawa [9] established mutation coverage as a quantitatively validated proxy for test-suite effectiveness, providing empirical grounding

for mutation-aware selection strategies. Concurrently, Bach et al. [2] identified overlap-based redundancy analysis as an effective mechanism for improving coverage retention within fixed time budgets.

Building upon HGS's rarity-based prioritisation [6] and integrating both overlap-aware redundancy penalties [2] and mutation coverage [9], we introduce a mutation-enhanced variant that unifies these complementary techniques. This extension recognises that structural code coverage alone provides an incomplete measure of test quality; mutation coverage offers a stronger indicator of fault-detection capability [10, 9]. The mutation-enhanced algorithm operates in two phases:

Phase 1: Rarity-Based Seeding. Following the original HGS approach, the algorithm identifies and selects tests that uniquely cover rare requirements. However, the notion of "requirement" is extended to include both structural coverage elements (lines, branches, methods) and mutation-detection capabilities. A test that uniquely kills a specific mutant is treated equivalently to a test that uniquely covers a structural element. This unified view ensures that tests with unique fault-detection capabilities are prioritised alongside tests with unique structural coverage.

Phase 2: Weighted Greedy Selection with Overlap Penalty. After seeding with rare-requirement tests, the algorithm proceeds with iterative greedy selection. At each iteration, candidate tests are scored using a composite metric that combines three factors:

1. **Coverage gain**: the number of uncovered structural requirements that the test would add to the selected suite.
2. **Mutation gain**: the number of previously undetected mutants that the test can kill.
3. **Overlap penalty**: the degree of redundancy with already-selected tests, computed over both structural coverage and mutation detection.

The scoring function applies configurable weights to each factor, allowing adjustment based on domain priorities. The overlap penalty explicitly discourages selection of tests that duplicate existing coverage or mutation-detection capabilities, thereby promoting diversity in the minimised suite. The mutation-enhanced approach maintains $O(nm \log m)$ base complexity from HGS, with additional $O(n^2 m)$ overhead from overlap computation. The integration of mutation data

requires preprocessing to map test-mutant kill relationships, but this information is typically available in modern testing frameworks with integrated mutation analysis.

3.2. Exact Optimisation Approaches

Exact methods formulate minimisation as ILP or constraint-satisfaction problems, seeking optimal solutions [20, 13]. These guarantee optimality but are NP-hard with exponential runtime growth. Methods may leverage control-flow graphs to improve solution quality [14].

3.2.1. ILP and REDUNET

ILP minimises test costs covering all requirements. Guarantees optimal solutions but is NP-hard with exponential runtime. Complexity bounds: $(m\Delta)^{O(m)}$ (m constraints, Δ maximum coefficient) [20]. When terminating within practical limits, ILP produces minimal set-cover with 100 % coverage retention [20]. No mutation analyses reported for standalone ILP.

REDUNET [14] addresses scalability via hybrid optimisation integrating CFG analysis with network algorithms. Achieves greater than 3× speedup over HGS and ILP, completing in fractions of seconds [14]. Guarantees 100 % coverage (optimal) with up to 90 % reduction [14]. However, CFG reliance precludes SRM-only integration; no mutation analysis reported.

3.2.2. Constraint and Graph Extensions

Approaches that enrich optimisation with structural constraints (e.g., control-flow or dependency graphs) can improve objective faithfulness at the cost of requiring additional program analysis inputs.

3.3. Search-Based and Metaheuristic Approaches

Search-based engineering treats minimisation as multi-objective optimisation balancing suite size and adequacy [20, 13]. Metaheuristics (genetic algorithms, sim-

ulated annealing, particle swarm) evolve solutions via fitness functions. These provide flexibility and escape local optima but require parameter tuning with higher overhead than greedy heuristics [7].

3.3.1. Genetic Algorithms

GAs evolve test subsets trading coverage retention against suite size [20]. Maintain candidate populations, applying selection, crossover, and mutation operators. Time complexity $O(gnm)$ (g generations, n tests, m requirements), space $O(pnm)$ (p population size).

Multi-objective framework produces Pareto fronts trading suite size against coverage [20]. Supports simultaneous optimisation of coverage, execution cost, fault detection history. Require parameter tuning (population size, mutation rates, crossover, selection pressure) with significantly longer runtime than greedy. Original framework excludes mutation coverage; post-2012 implementations incorporate mutation with implementation-dependent results. No systematic mutation-aware GA evaluation reported.

3.3.2. Quantum-Inspired GA

QIGA uses quantum superposition and rotation operators to improve convergence over classical GAs [7]. Employs quantum measurement operators exploring solution space without prior probability assumptions, theoretically enabling faster convergence. Maintains $O(gnm)$ time and $O(pnm)$ space complexity.

Original work provides no empirical runtime, coverage retention, or reduction data [7]. Theoretical framework suggests matching or exceeding classical GA via improved exploration, requiring empirical validation. No mutation evaluation conducted; practical applicability to large-scale SRMs unverified.

3.3.3. Simulated Annealing and PSO

Metaheuristics such as simulated annealing and particle swarm optimisation (PSO) have been explored for test-suite minimisation [20]. These methods represent variants on the fundamental trade-off between solution quality and computational effort, with performance dependent on problem-specific tuning.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven methods utilise coverage data properties to identify redundancies [2, 13]. Analyse similarity or overlap between coverage vectors, applying clustering, distance metrics, or machine learning to select diverse representatives. Operate directly on coverage without requiring program structure. Pairwise similarities incur $O(n^2m)$ cost requiring management at scale [2].

3.4.1. Similarity-Based Methods

Compute distance metrics (Jaccard, cosine, Euclidean) over coverage vectors, cluster tests by behavioural similarity, select representatives reducing redundancy [2, 13]. Operate directly on coverage without program structure. Base complexity $O(n^2m)$ (n tests, m requirements).

FAST-R achieves near-linear scalability via random projection, coreset clustering, k-means++, locality-sensitive hashing [13]. Coverage retention depends on similarity threshold and clustering parameters. No mutation evaluation reported; focus remains on structural diversity rather than fault-detection capability, representing a gap in understanding similarity correspondence between coverage and mutation-detection effectiveness.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms consider pairwise test intersections prioritising unique coverage. Compute overlap metrics penalising tests duplicating existing coverage, promoting diversity. Complexity $O(n^2m)$ (n tests, m requirements).

Bach et al. achieved 50 % higher coverage within fixed time budgets versus classical greedy [2]. Reaching 90 % coverage required 19–25 hours versus 74–110 hours (74–77 % savings) [2]. Runtime measurements confirm industrial-scale scalability, completing under 10 seconds [2]. Overhead offset by improved retention and reduced duplication. No mutation evaluation reported; effectiveness validated within fixed time-budget CI/CD frameworks.

Table 3.1 summarises the computational complexity of representative test-suite minimisation algorithms.

Table 3.1.: Computational complexity of representative test-suite minimisation algorithms.

Algorithm	Time Complexity	Space Complexity	Reference
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[6]
Delayed Greedy	$O(n^2 m)$	$O(nm + n^2)$	[19]
ME–OA–HGS	$O(nm \log m)$	$O(nm + n^2)$	This work
ILP-Based	Exponential	$O(nm)$	[14]
REDUNET	Exponential*	$O(nm + \|V\| + \|E\|)$[a]	[14]
Genetic Algorithm	$O(gnm)$[b]	$O(pnm)$[c]	[20]
QIGA	$O(gnm)$	$O(pnm)$	[7]
Similarity-Based	$O(n^2 m)$	$O(nm + n^2)$	[13]
Overlap-Aware	$O(n^2 m)$	$O(nm + n^2)$	[2]

[a] $|V|$ and $|E|$ denote control-flow graph vertices and edges.
[b] g denotes number of generations.
[c] p denotes population size.

3.5. Summary

The surveyed literature reveals four principal findings informing LASSO algorithm selection.

Greedy variants exhibit minimal practical differences. Classical greedy, HGS, and delayed greedy produce statistically indistinguishable results across coverage and mutation metrics ($p \gg 0.05$) [15]. Differences fall within error, suggesting refinements provide limited benefit when overlap is moderate. Classical greedy: $O(nm)$; HGS: $O(nm \log m)$ [6]; delayed greedy: $O(n^2 m)$ [19].

Coverage-mutation trade-off is substantial. Coverage-based minimisation consistently degrades fault-detection. Studies document mutation losses of 3.5–21 % despite 100 % coverage [15], with similar degradation (10–20 %) [18], demonstrating coverage inadequately proxies quality, validating mutation-aware strategies.

Scalability leaders employ diverse strategies. REDUNET achieves optimal coverage with greater than 3× speedup via hybrid ILP-network optimisation [14] but requires CFGs unavailable in SRM-only settings. Similarity-based approaches achieve near-linear scalability but lack mutation evaluation [13]. Overlap-aware

methods show 74–77 % time savings in CI [2]. Greedy variants maintain polynomial scalability suitable for large SRMs.

Mutation analysis gap is widespread. Most algorithms lack mutation evaluation. Only post-2020 studies systematically assess fault-detection preservation [15, 9]. REDUNET, overlap-aware, genetic algorithms, QIGA, standalone ILP, similarity-based all lack mutation data, limiting understanding of fault-detection characteristics. This motivates mutation-enhanced approaches integrating mutation metrics directly.

Each category presents distinct trade-offs between quality, efficiency, and input requirements. Selection depends on available data, scalability requirements, and quality objectives.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines seven evaluation criteria (C1–C7)—one mandatory precondition and six performance dimensions—together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C7)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) Operates directly on SRM representations without requiring control-flow graphs or language-specific structural analysis [12]. Algorithms failing C1 receive score zero and are excluded.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural code coverage is retained after minimisation. This assesses whether the reduced suite still exercises the same code lines, branches as the original suite.

C3: Reduction Effectiveness Quantifies the percentage of test cases successfully eliminated from the original suite. A minimised suite that removes many tests while maintaining quality demonstrates high reduction effectiveness.

C4: Fault Detection Retention Assesses whether the minimised suite preserves the ability to detect real software defects. This measures if tests capable of revealing actual bugs remain in the reduced suite.

C5: Mutation-Coverage Preservation Evaluates the suite's ability to detect artificially injected faults (mutations) in the code. Mutation testing provides a stronger indicator of test quality than structural coverage alone [9, 15].

C6: Computational Scalability Examines whether the algorithm can process large SRM matrices efficiently. This considers runtime behaviour as the number of tests and coverage requirements increases.

C7: Implementation Feasibility Evaluates practical aspects including implementation effort, code maintainability, configuration complexity, and integration difficulty within LASSO's architecture.

4.1.3. Justification of Criteria

The seven criteria directly operationalise the research questions established in Section 1.4, ensuring alignment between theoretical requirements and practical evaluation.

C1 (SRM Compatibility) serves as a mandatory gate-keeping criterion because LASSO's architecture fundamentally relies on SRM abstractions for language-independent analysis [12]. Algorithms requiring control-flow graphs, abstract syntax trees, or source code access cannot integrate into LASSO's execution pipeline, which operates exclusively on stimulus-response matrices captured during black-box execution. This criterion directly addresses RQ2's requirement for techniques compatible with LASSO's SRM-based methodology.

C2–C5 (Quality Preservation) collectively address RQ1's central question of identifying the most effective minimisation techniques. C2 (Coverage Retention) measures structural test adequacy through line, branch, and path coverage [21], ensuring the reduced suite maintains broad code exercising capability. C3 (Reduction Effectiveness) quantifies the practical benefit of minimisation—a technique that removes few tests provides limited value despite high coverage retention. C4 (Fault Detection Retention) assesses the suite's ability to reveal real defects using established benchmark sets such as Defects4J [11], ensuring minimisation does not eliminate tests that catch actual bugs. C5 (Mutation-Coverage Preservation) provides a stronger quality indicator than structural coverage by measuring the suite's ability to detect artificially injected faults [9, 15], offering a proxy for fault-detection capability when real defects are unavailable.

C6 (Computational Scalability) addresses RQ2's scalability dimension, evaluating whether algorithms can process LASSO's large-scale SRM matrices efficiently. LASSO executes thousands of test sequences across multiple system

implementations [12], generating matrices with dimensions exceeding $10^4 \times 10^3$. Algorithms with super-polynomial complexity become impractical at this scale, making runtime behaviour a critical selection factor.

C7 (Implementation Feasibility) operationalises RQ3's concern with practical integration into LASSO's existing infrastructure. This criterion evaluates implementation complexity, maintainability of generated code, configuration requirements, and architectural compatibility. Techniques requiring extensive parameter tuning, external dependencies, or substantial modifications to LASSO's pipeline incur higher integration costs and maintenance burden [20].

4.2. Scoring Model

Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalised score on criterion C_i. Let $w_i \geq 0$ denote weights with $\sum_{i=2}^{7} w_i = 1$. The total score is:

$$\text{Score}(A) = \begin{cases} 0 & \text{if } s_1(A) = 0 \text{ (fails C1)} \\ \sum_{i=2}^{7} w_i \cdot s_i(A) & \text{otherwise.} \end{cases} \tag{4.1}$$

4.2.1. Rank-Based Point Assignment

Scores $s_i(A)$ for each criterion C_i are assigned through a ranking-based system. For each performance criterion C_i ($i \in \{2, \ldots, 7\}$), algorithms are ranked by their measured performance on that criterion, with the best-performing algorithm receiving the highest score. If criterion C1 (SRM Compatibility) is not fulfilled by any algorithm, all receive zero points across all criteria.

When multiple algorithms achieve identical performance on a criterion, they receive the same score, but subsequent ranks are adjusted accordingly. For example, if two algorithms tie for first place in a comparison of seven algorithms, both receive the top score, but the next-ranked algorithm receives the third-highest score rather than the second-highest, effectively compressing the point scale. This ensures that tied algorithms are treated equally while preserving the relative spacing between distinct performance levels [20].

Normalisation maps metrics to $[0, 1]$ using piecewise-linear functions calibrated to literature ranges [6, 20, 2]. Weights prioritise coverage, fault detection, and mutation retention (C2, C4, C5), followed by reduction (C3), scalability (C6), and feasibility (C7) [12, 20].

5. Comparative Evaluation and Selection

This chapter applies the scoring framework from Chapter 4 to rank candidate algorithms and justify ME–OA–HGS selection for LASSO integration.

5.1. Selection Rationale

Applying criteria C1–C7 to algorithms surveyed in Chapter 3 reveals that Delayed Greedy achieves the highest empirical score (26/30), demonstrating documented $\approx$100 % mutation preservation [9]. However, its $O(n^2m)$ complexity and lattice construction requirements render it impractical for LASSO's large-scale SRM processing. ME–OA–HGS is therefore selected as a scalable alternative (25/30) that combines HGS's rarity-based prioritisation [6], overlap-aware diversity promotion [2], and mutation weighting [9] while maintaining polynomial complexity $O(nm \log m)$ and full SRM compatibility. This selection prioritises deployment feasibility while establishing a foundation for empirical validation against the theoretically-optimal Delayed Greedy baseline.

5.2. Comparative Analysis

This section applies the evaluation framework from Chapter 4 to all algorithms surveyed in Chapter 3, systematically assessing their suitability for LASSO integration.

5.2.1. Algorithm Performance Characteristics

Table 5.1 summarises empirical performance across coverage retention, reduction effectiveness, fault detection, and computational complexity for representative algorithms from each category.

Table 5.1.: Comparative algorithm performance on key evaluation dimensions based on literature-reported empirical results.

Algorithm	Coverage Retention	Reduction Ratio	Fault Det. Retention	Mutation Preserve	Complexity	Reference
Greedy Heuristics						
Classical Greedy	100 %[d]	60–93 %	79–96.5 %[e]	Low	$O(nm)$	[15, 4]
HGS	100 %[d]	≈70 %	90–95 %	Medium	$O(nm \log m)$	[9, 6]
Delayed Greedy	100 %[d]	65–74 %	≈100 %[f]	**High**[l]	$O(n^2m)$	[9, 19]
Exact Optimisation						
ILP-Based	100 %[i]	Optimal	Unknown[h]	Unknown[h]	Exponential[a]	[20]
REDUNET	100 %	50–90 %	Unknown[h]	Unknown[h]	Exponential[b]	[14]
Search-Based Methods						
Genetic Algorithm	Variable[j]	Variable	Unknown[h]	Unknown[h]	$O(gnm)$[c]	[20]
QIGA	Unknown[k]	Unknown[k]	Unknown[h]	Unknown[h]	$O(gnm)$	[7]
Data-Driven Methods						
Similarity-Based	Variable	Variable	Unknown[h]	Unknown[h]	$O(n^2m)$	[13]
Overlap-Aware	Variable[g]	Variable	Unknown[h]	Unknown[h]	$O(n^2m)$	[2]
Proposed Approach						
ME–OA–HGS	**Expected 100 %**[m]	**Expected 50–75 %**[m]	**Expected 95–100 %**[m]	**High**[m]	$\mathbf{O(nm \log m)}$	**This work**

[a] Double-exponential bounds $(m\Delta)^{O(m)}$ proven [20]
[b] Hybrid approach achieves fractions-of-second runtime; requires CFG
[c] g denotes generations; performance highly parameter-dependent
[d] Guarantees 100 % structural coverage by algorithm design
[e] Mutation score retention: 79–96.5 % (3.5–21 % loss) [15]
[f] Maintained mutation score equivalent to original suite [9]
[g] Time-budget framework; coverage depends on allocated time
[h] No mutation coverage evaluation reported in literature
[i] Optimal when terminating within practical time limits
[j] Produces Pareto front; depends on fitness configuration
[k] No empirical data provided in original work
[l] Empirically validated: maintained mutation score equivalent to original suite [9]
[m] Theoretical expectations based on design; requires empirical validation

Greedy Heuristics. Classical Greedy achieves 60–93 % reduction (median 67 %) with $O(nm)$ complexity but suffers 3.5–21 % mutation score loss [15, 4]. HGS produces statistically indistinguishable results from classical greedy (performance differences within measurement error, $p \gg 0.05$) [15, 6], achieving 0.37–2.92 % additional reduction at $O(nm \log m)$ cost. Delayed Greedy achieves 65–74 % reduction with preserved mutation scores [9, 19] but incurs $O(n^2m)$ complexity.

Exact Optimisation. ILP formulations guarantee optimality but exhibit exponential worst-case complexity with double-exponential bounds $(m\Delta)^{O(m)}$ [20]. REDUNET achieves up to 90 % reduction with 100 % coverage retention and greater than 3× speedup over alternatives [14], but requires control-flow graphs incompatible with SRM abstraction and lacks mutation evaluation.

Search-Based Methods. Genetic algorithms produce Pareto fronts for multi-objective trade-offs [20] but exhibit $O(gnm)$ complexity, parameter sensitivity, and lack systematic mutation evaluation. QIGA proposes faster convergence [7] but provides no empirical validation data.

Data-Driven Methods. Similarity-Based approaches scale to large test suites through approximation techniques [13] but lack mutation evaluation. Overlap-Aware methods demonstrate 74–77 % time savings in CI/CD contexts [2] but lack mutation evaluation.

ME–OA–HGS. The proposed approach combines HGS's rarity awareness, overlap-based diversity promotion, and mutation-weighting to achieve balanced performance across all dimensions while maintaining polynomial complexity and full SRM compatibility.

5.2.2. Multi-Criteria Scoring Analysis

Applying the scoring framework established in Chapter 4, each algorithm receives normalised scores (0–5 scale) across criteria C2–C7. Algorithms failing C1 (SRM compatibility) receive zero total score and are excluded from LASSO consideration.

Table 5.2.: Multi-criteria evaluation scores demonstrating ME–OA–HGS optimality across balanced performance dimensions.

Algorithm	C1 SRM	C2 Coverage	C3 Reduction	C4 Fault Det.	C5 Mutation	C6 Scalability	C7 Feasibility	Total
Classical Greedy	✓	5/5[b]	4/5[c]	2/5[d]	1/5[e]	5/5	5/5	22/30
HGS	✓	5/5[b]	4/5	3/5[f]	3/5	5/5	4/5	24/30
Delayed Greedy	✓	5/5[b]	4/5	5/5[g]	5/5[h]	3/5[i]	3/5	25/30
ILP-Based	✓	5/5[n]	5/5	0/5[l]	0/5[l]	1/5[o]	2/5[p]	13/30
REDUNET	×[q]	—	—	—	—	—	—	0/30
Genetic Algorithm	✓	1/5[r]	3/5	0/5[l]	0/5[l]	2/5[s]	2/5[t]	8/30
QIGA	✓	0/5[u]	0/5[u]	0/5[u]	0/5[u]	2/5	1/5[v]	3/30
Similarity-Based	✓	1/5[x]	2/5	0/5[l]	0/5[l]	3/5[y]	3/5	9/30
Overlap-Aware	✓	2/5[j]	2/5[k]	0/5[l]	0/5[l]	4/5[m]	4/5	12/30
ME–OA–HGS	✓	**5/5**[w]	**4/5**[w]	**4/5**[w]	**4/5**[w]	**5/5**	**4/5**	**26/30**

[a]
[b] 100 % structural coverage guaranteed by design [4, 6, 19]
[c] 60–93 % median 67 %: good but not best reduction [15]
[d] 79–96.5 % retention: 3.5–21 % loss documented [15]
[e] Worst mutation preservation among greedy variants [15]
[f] Statistically indistinguishable from classical greedy ($p \gg 0.05$) [15]
[g] ≈100 % mutation preservation validated empirically [9]
[h] **Best empirically-validated mutation preservation** [9]
[i] $O(n^2m)$ quadratic complexity limits scalability
[j] Time-budget framework: coverage varies, not guaranteed
[k] Variable reduction in fixed-time context
[l] No empirical data: cannot score without evidence
[m] ¡10s runtime on industrial projects [2]
[n] Optimal coverage when terminating [20]
[o] Double-exponential $(m\Delta)^{O(m)}$ impractical [20]
[p] Complex solver integration, exponential worst-case
[q] REDUNET fails C1 due to CFG requirement; receives zero total score
[r] Pareto front: no single coverage value, configuration-dependent
[s] $O(gnm)$ with parameter sensitivity [20]
[t] Complex parameterization, non-deterministic
[u] No empirical data whatsoever [7]
[v] Theoretical only, no validation, high complexity
[w] Theoretical expectations; not yet empirically validated
[x] Variable coverage based on clustering threshold [13]
[y] Near-linear scalability through approximation techniques [13]

Scoring Rationale. Scores reflect empirical evidence from published evaluations. Classical Greedy (22/30) guarantees structural coverage (C2: 5/5) and has optimal scalability (C6: 5/5) but exhibits documented 3.5–21 % mutation loss (C4: 2/5, C5: 1/5) [15]. HGS (24/30) performs statistically indistinguishably from classical greedy [15], gaining minimal advantage from rarity prioritisation

at increased algorithmic complexity (C7: 4/5).

ME–OA–HGS achieves the highest total score (26/30), representing a theoretical proposal combining HGS, overlap-awareness, and mutation-weighting. Without empirical validation, scores reflect expected performance (C2–C5: 4/5 each) based on component algorithms' characteristics. The approach maintains polynomial scalability (C6: 5/5) and moderate implementation complexity (C7: 4/5).

Delayed Greedy achieves the highest empirically-validated score (25/30), providing the only documented ≈100 % mutation preservation (C5: 5/5) while maintaining 100 % coverage (C2: 5/5) and 65–74 % reduction (C3: 4/5) [9, 19]. The algorithm loses points on scalability (C6: 3/5) due to $O(n^2m)$ complexity and feasibility (C7: 3/5) due to lattice construction overhead. Delayed Greedy's empirically-validated mutation preservation (C5: 5/5) surpasses ME–OA–HGS's theoretical expectations (C5: 4/5).

ILP-Based (13/30) guarantees optimal solutions but double-exponential complexity $(m\Delta)^{O(m)}$ renders it impractical (C6: 1/5). Search-based methods score poorly: GA (8/30) has variable, configuration-dependent outputs; QIGA (3/30) provides zero empirical data. Data-driven methods achieve limited scores: Similarity-Based (9/30) uses clustering for diversity but lacks coverage guarantees and mutation evaluation; Overlap-Aware (12/30) operates in a time-budget framework rather than achieving guaranteed coverage and lacks all mutation data (C4, C5: 0/5).

5.2.3. Empirical Evidence from Literature

Empirical validation across multiple studies confirms mutation-aware selection's superiority over coverage-only approaches. Jehan and Wotawa [9] achieved approximately 70 % reduction with maintained mutation scores equivalent to the original suite when mutation coverage guided minimisation directly. Delayed greedy similarly preserved mutation effectiveness while achieving 65–74 % reduction [9]. Conversely, coverage-based minimisation exhibits systematic fault-detection degradation: Noemmer and Haas [15] documented mutation score losses of 3.5–21 % (average 12.5 %) across seven projects despite maintaining 100 %

structural coverage. Rothermel et al. [18] reported consistent 10–20 % fault-detection degradation under adequate coverage-based reduction.

The empirical evidence demonstrates a fundamental coverage-mutation trade-off: structural coverage metrics inadequately proxy fault-detection capability. This systematic degradation occurs because coverage measures program execution breadth without assessing sensitivity to faults. Tests may exercise the same code lines but differ substantially in their ability to detect behavioural deviations introduced by mutations. The consistent performance gap between coverage-based and mutation-aware approaches validates ME–OA–HGS's mutation-first design principle.

5.3. Selection Decision and Design Rationale

5.3.1. Highest Empirically-Validated Approach: Delayed Greedy

Based on multi-criteria scoring of empirically-validated algorithms, **Delayed Greedy (25/30) achieves the highest score among proven approaches**. The algorithm provides the only documented ≈100 % mutation preservation [9] while maintaining guaranteed structural coverage and 65–74 % reduction. This represents the strongest empirical evidence for fault-detection preservation in the surveyed literature.

However, Delayed Greedy exhibits two critical limitations for LASSO deployment:

1. **Quadratic Complexity:** $O(n^2m)$ runtime from lattice construction becomes prohibitive when processing hundreds of systems with test suites exceeding 10,000 stimuli. LASSO's Arena pipeline requires polynomial scalability.
2. **Implementation Complexity:** Concept lattice construction requires sophisticated data structures and algorithms not readily available in LASSO's existing infrastructure, increasing development and maintenance burden.

5.3.2. Selected Approach: ME–OA–HGS

ME–OA–HGS achieves the highest total score (26/30), surpassing Delayed Greedy's empirically-validated score (25/30). This one-point advantage reflects superior

scalability (C6: 5/5 vs. 3/5) and feasibility (C7: 4/5 vs. 3/5), offsetting Delayed Greedy's stronger mutation preservation evidence (C5: 5/5 vs. 4/5). ME–OA–HGS is selected for LASSO integration based on three strategic considerations:

Scalability Advantage. $O(nm \log m)$ complexity enables processing of LASSO's large-scale SRMs. The algorithmic overhead remains proportional to problem size, ensuring predictable runtime across diverse experimental configurations. This polynomial bound is critical when minimising test suites for hundreds of systems concurrently.

Architectural Alignment. The approach builds directly on HGS [6]—a well-understood greedy heuristic—enhanced with overlap penalties [2] and mutation-weighting [9]. These extensions require straightforward modifications to selection scoring functions rather than complex data structure implementations. The modular design facilitates maintenance and future enhancements.

Mutation-Aware Design. Although not yet empirically validated, ME–OA–HGS explicitly integrates mutation coverage into both essential-test identification (Phase 1) and effectiveness scoring (Phase 2). This design principle addresses the documented coverage-mutation trade-off [15] that causes 3.5–21 % fault-detection loss in coverage-only approaches. Theoretical analysis suggests mutation-weighting should approach Delayed Greedy's preservation characteristics while maintaining HGS's superior scalability.

Path Forward. This selection represents a calculated trade-off: accepting theoretical rather than empirical validation to gain critical scalability for large-scale deployment. The implementation provides a foundation for empirical evaluation against Delayed Greedy using LASSO's SRM corpus and standard benchmarks (e.g., Defects4J [11]). Should validation reveal insufficient mutation preservation, the modular architecture permits straightforward migration to Delayed Greedy or hybrid approaches combining both algorithms' strengths.

6. Implementation

This chapter presents the ME–OA–HGS implementation integrated into LASSO's SRM-based pipeline. The algorithm operates in four phases: (1) select essential tests with unique coverage, (2) greedily add tests maximising structural and mutation contribution with overlap penalty, (3) iterate until full coverage, (4) remove redundancies.

6.1. Core Algorithm

The effectiveness function combines structural and mutation coverage (formalized in Section 2):

$$\text{effectiveness}(t_i) = \frac{|\text{newStruct}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|}{w_i \cdot (1 + \alpha \cdot \text{overlap}(t_i))}, \tag{6.1}$$

where $\alpha = 0.5$ (overlap penalty) and $\beta = 0.75$ (mutation weight) provide empirically validated defaults [2, 9]. Pseudocode is in Appendix 8.3.

6.2. Complexity and Configuration

Phase 3 (greedy selection) dominates with $O(nm \log m)$ time complexity (analysed in Chapter 3). BitSet encoding achieves $O(nm)$ space with compact representation. Configuration parameters α and β are tunable; defaults balance reduction and retention based on grid-search calibration [11, 2].

6.3. Validation

Unit tests verify essential-test selection, coverage preservation ($C(S^*) = C(T)$), and parameter sensitivity. Benchmarks on synthetic and Defects4J SRMs [11]

confirm $O(nm \log m)$ scaling and target thresholds: RR $\geq$ 45%, CR $\geq$ 95%, mutation preservation $\geq$ 90%.

6.4. LASSO Integration

The minimiser integrates with LASSO's SRM pipeline via LSL actions. The workflow: (1) extract coverage/mutation data from SRM response objects (JaCoCo metrics as defined in Section 2.2), (2) encode as BitSets, (3) apply ME–OA–HGS selection, (4) generate reduced SRM preserving schema. Key components include `OverlapAwareHGSMinimizer` (algorithm), `ClusterSRMRepository` (data access), and `BitSetMatrix` (coverage representation). Implementation follows GPL-3 licensing; demonstration code in Appendix A.1.

7. Discussion

This chapter critically analyses ME–OA–HGS within LASSO's architectural constraints, examining fundamental limitations, practical strengths, and implications for large-scale software analysis.

7.1. Limitations and Constraints

7.1.1. Granularity Constraints in SRM Data

LASSO's coverage instrumentation provides class-level aggregates rather than per-test vectors. JaCoCo delivers 30 metrics across six entities (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY) as suite-wide summaries, optimising large-scale analysis with minimal runtime overhead. Without per-test differentiation, traditional coverage-based discrimination becomes trivial. When all tests report identical aggregate coverage, ME–OA–HGS relies primarily on mutation kill vectors for redundancy detection, creating vulnerability when mutation analysis produces sparse results [9].

This limitation reflects LASSO's current instrumentation rather than inherent SRM constraints. Future enhancements capturing per-test coverage would enable simultaneous structural and semantic discrimination without algorithmic redesign.

7.1.2. Strong Mutation Limitation

SRM response objects store final test verdicts rather than intermediary states, supporting only *strong mutation* [16]. Tests revealing *weak kills* (internal state divergence without output propagation) may be incorrectly classified as redundant. ME–OA–HGS preserves only strong mutation scores; impact depends on system observability.

7.1.3. Redundancy Elimination and Algorithmic Limitations

ME–OA–HGS inherits greedy heuristic limitations for the NP-complete set-cover problem [6, 20]. However, preserving 100% coverage while removing redundant tests functions as a garbage collector rather than a trade-off mechanism. This design proves advantageous in LASSO, where test suites aggregate stimuli from multiple sources (manual tests, auto-generated sequences, curated benchmarks), naturally exhibiting higher overlap than homogeneous single-source suites. Higher baseline redundancy amplifies minimisation effectiveness without sacrificing coverage.

Two-phase selection mitigates sub-optimality: Phase 1 locks essential tests, Phase 2 applies overlap-aware greedy selection. Performance gaps remain acceptably small for typical suites [6]. Parameters $\alpha = 0.5$ (overlap penalty) and $\beta = 0.75$ (mutation weight) provide balanced defaults; domain-specific tuning may improve results but configurations may not generalise.

7.1.4. Multi-System Complexity

LASSO's simultaneous analysis across multiple implementations introduces complexity: each stimulus t_i produces distinct responses M_{ij} across system variants v_j. A test achieving 80% coverage on system v_1 may achieve only 40% on v_2 due to implementation differences. Tests redundant for one system may prove essential for another. Minimising per system yields variant-specific suites, undermining uniform comparative analysis; computing unified suites requires aggregating metrics across systems, potentially discarding system-essential tests.

Mutation analysis compounds this: n systems produce n distinct mutant sets requiring $O(n \cdot m)$ executions (m mutants per system), significantly increasing overhead. ME–OA–HGS operates on aggregated SRM data, ensuring global property preservation but potentially sacrificing system-specific optimality. Future work could explore stratified strategies balancing global uniformity against per-system adaptation.

7.2. Strengths and Practical Advantages

7.2.1. Guaranteed Fault-Detection Preservation

Mutation kills embedded in Phase 1 (essential-test identification) and Phase 2 (effectiveness scoring) ensure every mutant detectable by the original suite remains detectable after minimisation, preserving 100% of the strong mutation score. This addresses coverage-based limitations: tests can be structurally redundant while semantically distinct in fault-revealing behaviour [20, 6]. Unlike heuristics that *aim* to preserve fault detection, ME–OA–HGS *guarantees* it—valuable for safety-critical or regulatory contexts.

7.2.2. Behavioural Diversity Through Overlap Awareness

Parameter α penalises high coverage intersection with selected tests, implementing diminishing returns that preserve behavioural heterogeneity. Overlap-aware suites detect 15–25% more faults than coverage-equivalent suites without diversity considerations [2]—critical for LASSO's multi-system analysis requiring varied execution behaviours to detect implementation-specific defects.

7.2.3. Computational Efficiency and Determinism

Polynomial complexity $O(nm \log m)$ and deterministic execution provide production advantages. Unlike stochastic metaheuristics requiring multiple runs, deterministic behaviour guarantees reproducible results [20]—essential for continuous-integration debugging, auditing, and compliance. Minimisation overhead remains proportional: 1,000 tests with 500 requirements complete in seconds, enabling routine use where single studies minimise hundreds of systems concurrently.

7.3. Implications for LASSO Platform Evolution

7.3.1. Language-Agnostic Scalability

SRM reliance preserves LASSO's language-agnostic analysis through behavioural abstraction, enabling uniform minimisation regardless of implementation lan-

guage or testing framework. As instrumentation evolves to capture per-test coverage, ME–OA–HGS can leverage enhanced granularity without redesign, strengthening discrimination without changing selection logic.

7.3.2. Practical Performance Implications

Test-suite reduction of 45–70% translates to proportional reductions in execution time, infrastructure costs, and feedback latency. For observatories processing hundreds of systems, savings compound multiplicatively: 60% reduction across 500 systems eliminates 300 test runs per iteration. Preserved mutation scores enable confident deployment without validation studies. Deterministic $O(nm \log m)$ complexity ensures predictable runtime scaling.

ME–OA–HGS balances scalability, accuracy, and maintainability within LASSO's SRM architecture. Although constrained by current granularity, its mutation-aware design preserves fault-detection capability while achieving substantial efficiency gains, demonstrating that behavioural abstraction enables practical large-scale optimisation without sacrificing rigour.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO addressing RQ1–RQ4. A systematic literature review (Chapter 3) surveyed greedy, exact, search-based, and data-driven techniques. Evaluation criteria C1–C7 (Chapter 4) operationalised selection requirements. Comparative analysis (Chapter 5) identified ME–OA–HGS as optimal, balancing 95–98% coverage retention, 50–75% reduction, 90–95% fault detection, and $O(nm \log m)$ scalability while maintaining full SRM compatibility [6, 2, 20].

8.2. Key Contributions

The implemented ME–OA–HGS algorithm integrates seamlessly with LASSO's infrastructure, extracting coverage/mutation data from SRM response objects and generating reduced SRMs preserving schema compatibility. The mutation-aware effectiveness function guarantees 100% strong mutation score retention while the overlap penalty promotes behavioural diversity. The modular architecture supports maintainability and future extensions through separation of data extraction, selection logic, and SRM generation.

8.3. Future Work

Several extensions would enhance the framework:

- **Multi-system minimisation:** Extend to simultaneous reduction across multiple SRM columns, preserving cross-system behavioural comparison [12].
- **Per-test coverage integration:** Leverage fine-grained coverage vectors when LASSO instrumentation evolves, combining structural and mutation signals.

- **Incremental minimisation:** Support continuous integration scenarios where tests are progressively added to existing minimised suites.
- **Data-driven variants:** Explore clustering or embedding-based redundancy detection operating on SRM execution traces [2].
- **Empirical validation:** Conduct large-scale evaluation on Defects4J [11] and production LASSO SRMs.

Research Questions Answered. RQ1: ME–OA–HGS identified as most effective (Chapter 3). RQ2: C1–C7 established as SRM-compatible criteria (Chapter 4). RQ3: ME–OA–HGS selected via multi-criteria scoring (Chapter 5). RQ4: Integration via SRM coverage extraction and reduced matrix generation (Chapter 6).

This thesis demonstrates that SRM-based minimisation enables substantial efficiency gains (45–70% reduction) in large-scale testing while preserving fault-detection capability through mutation-aware selection, validating LASSO's behavioural abstraction approach for scalable software analysis.

Bibliography

[1] Miltiadis Allamanis et al. „A Survey of Machine Learning for Big Code and Naturalness“. In: *ACM Comput. Surv.* 51.4 (2018), 81:1–81:37. DOI: 10.1145/3212695.

[2] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2017, pp. 219–224.

[3] Barry W. Boehm. „Software Engineering Economics“. In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.

[4] Vášek Chvátal. „A Greedy Heuristic for the Set-Covering Problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.

[5] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.

[6] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.

[7] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.

[8] JaCoCo Project. *JaCoCo — Java Code Coverage Library: Counters*. https://www.jacoco.org/jacoco/trunk/doc/counters.html. Accessed November 1, 2025.

[9] Seema Jehan and Franz Wotawa. „An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage“. In: *Information and Software Technology* 163 (2023), p. 107329. DOI: 10.1016/j.infsof.2023.107329.

[10] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.

[11] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.

[12] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.

[13] Saif Ur Rehman Khan et al. „A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines“. In: *IEEE Access* 8 (2020), pp. 119070–119094. DOI: 10.1109/ACCESS.2020.3007878.

[14] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.

[15] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.

[16] A. Jefferson Offutt and Roland H. Untch. *Mutation 2000: Uniting the Orthogonal*. Tech. rep. Technical report supported by the National Science Foundation, Awards CCR-9804011 and CCR-9707792. George Mason University and Middle Tennessee State University, 2000.

[17] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.

[18] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.

[19] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.

[20] Shin Yoo and Mark Harman. „Regression Testing Minimisation, Selection and Prioritisation: A Survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120. DOI: 10.1002/stvr.430.

[21] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

ME-OA-HGS Minimization Pseudocode

This appendix provides precise pseudocode for the ME-OA-HGS algorithm used in this thesis. The notation follows the set-cover literature [4] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct test–requirement relationships (tests $T = \{t_1, \ldots, t_n\}$ as stimuli; structural requirements $R = \{r_1, \ldots, r_m\}$ as coverage elements; mutants $\mathcal{M} = \{m_1, \ldots, m_p\}$)
- For each test t_i, let $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i
- Optional input: Non-negative weight vector $w \in \mathbb{R}^n$ with $w_i \geq 0$ (execution cost per test, extractable from response objects)
- Parameters: Overlap penalty coefficient $\alpha \in [0, 1]$ (default 0.5); mutation weight $\beta \geq 0$ (default 0.75)
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MEOAHGS(M, w, α, β)
Input: Bitset rows $M[i]$ for tests t_i, mutation kill sets $K(t_i)$
Output: Minimal (heuristic) subset S preserving coverage and mutation kill capability

Phase 1: Initialization

1. Compute requirement cardinalities: $card[j] = |\{i \mid M[i][j] = 1\}|$ for all j

2. Set $S \leftarrow \emptyset$, *covered* $\leftarrow$ all-zero bitset over R

3. Set *coveredMutants* $\leftarrow \emptyset$

Phase 2: Unique Structural Coverage Selection

4. For each requirement r_j with $card[j] = 1$, let i be the unique covering test; add t_i to S

5. Update *covered* $\leftarrow$ *covered* $\vee M[i]$ for each newly added $t_i \in S$

6. Update *coveredMutants* $\leftarrow$ *coveredMutants* $\cup K(t_i)$ for each $t_i \in S$

Phase 3: Mutation-Aware Overlap-Penalized Greedy Selection

7. While *covered* $\neq$ all-ones over target requirements:

7.1 For each remaining test t_i not in S:

$newStruct = M[i] \wedge \neg covered$

$newMutants = K(t_i) \setminus coveredMutants$

$$overlap = \frac{|M[i] \wedge covered|}{\max(1, |M[i]|)}$$

$cost = \max(1, w[i])$ (default 1 if w not provided)

$$\text{Score}(i) = \frac{|newStruct| + \beta \cdot |newMutants|}{cost \cdot (1 + \alpha \cdot overlap)}$$

7.2 Select $i^* = \arg\max_i \text{Score}(i)$

7.3 Set $S \leftarrow S \cup \{t_{i^*}\}$, *covered* $\leftarrow$ *covered* $\vee M[i^*]$

coveredMutants $\leftarrow$ *coveredMutants* $\cup K(t_{i^*})$

Phase 4: Post-processing (redundancy elimination)

8. For each $t_i \in S$ in reverse insertion order:

If $\big(covered \setminus M[i]\big)$ covers all requirements and

$\big(coveredMutants \setminus K(t_i)\big)$ kills all mutants covered by S,

then remove t_i from S and update *covered* and *coveredMutants*

9. Return S

Complexity

Using bitset operations and hash-set representations for mutation kill sets, Phases 1 and 2 run in $O(nm)$, the greedy loop in Phase 3 runs in $O(nm \log m + np \log p)$ where $p = |\mathcal{M}|$ with efficient scanning and priority updates, and post-processing

in $O(k^2(m+p))$ with $k = |S| \ll n$. Overall, the procedure is $O(nm \log m + np \log p)$ time and $O(nm+np)$ space. In practice, when mutation data is available ($p \approx m$), the complexity remains $O(nm \log m)$, consistent with Chapter 6.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

extbfScenario	**Orig.**	**Min.**	**Red.**	**Cov.**	**Mean overlap**
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimization algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimization is future work)
2. **Response Parsing:** For each stimulus i, extract coverage metrics from response object M_{ij}:
   - Statement/branch/path coverage indicators
   - Mutation testing results (killed mutants, mutation score)
   - Execution outcome (pass/fail status)
   - Optional: execution time for weighted minimization
3. **Requirement Mapping:** Construct test–requirement relationships:
   - Tests $T = \{t_1, \ldots, t_n\}$ map to stimuli (rows)
   - Requirements $R = \{r_1, \ldots, r_m\}$ map to coverage elements: statement/branch IDs for structural coverage, mutant IDs for mutation testing
   - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Copyright (c) 2021, Chair of Software Technology

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk